

DUCKIETOWN · TUM CAMPUS HEILBRONN

GentleStar

Safe navigation in Duckietown

An autonomous navigation stack for a Duckiebot: classical computer vision, weighted **A*** route planning and shape-based duck detection, running entirely **off-robot**.

TEAM

Yahia Taha · Ahmad Kaihan Saba · Đorđe Vidaković

INSTITUTION

Technical University of Munich

01 / 25

Who built it



Yahia Taha

YAHIA.TAHA@TUM.DE



Ahmad Kaihan Saba

KAIHAN.SABA@TUM.DE



Đorđe Vidaković

DJORDJE.VIDAKOVIC@TUM.DE

PROGRAMME

Information Engineering (B.Sc.)
TUM Campus Heilbronn

ROBOT

duck3, our assigned Duckiebot,
on the TUM Duckietown lab track

DELIVERABLE

A repeatable autonomous run from a
fixed start tile to a fixed goal tile

What we'll cover

01	The mission	Goal, constraints, architecture, state machine
02	Lane following	HSV perception, search gating, PD steering
03	Intersection navigation	A tile graph, weighted A*, executing a turn
04	Object detection & avoidance	Telling a duck from a lane line, then avoiding it
05	A negative result	183M steps of PPO, and why it never shipped
06	Results, limits, next steps	Where it stands, and what we would fix first

Get **duck3** across town, without touching a duck

Drive our Duckiebot from a fixed start tile to a fixed goal tile on the TUM lab track: **follow the lane, stop at red lines, take the right way through an intersection, and never touch a duck.**

- LANE FOLLOWING

- INTERSECTION NAVIGATION

- OBJECT AVOIDANCE

3

COOPERATING
CAPABILITIES

8

TILES FROM START
TO GOAL

0

LINES OF OUR CODE
ON THE ROBOT

One Python process on a laptop. Only stock Duckietown containers run on duck3.

Everything runs off the robot

One Python process on a laptop reads duck3's camera over a rosbridge WebSocket and publishes velocity commands back. The robot runs no custom code of ours.

LAPTOP · WSL2 / LINUX

navigate.py

Perception, steering, planning, state machine and the dashboard on :5000

OPENCV

NUMPY

ROSLIBPY

FLASK

ROSBRIDGE
WS :9001

DUCK3 · DUCKIEBOT

Stock containers only

duckiebot-interface, rosbridge-websocket and car-interface

CAMERA

WHEELS

ROS

IN ↓

/duck3/camera_node/image/compressed

JPEG FRAMES · ~30 FPS

OUT ↑

/duck3/car_cmd_switch_node/cmd

TWIST2DSTAMPED · SPEED + TURN RATE

One home state, four detours

Every frame is classified, then one state machine decides what the wheels do. Four of the five below are temporary detours back to lane following; the fifth ends the run.

HOME STATE

lane_following

Vision-based steering. Every detour returns here.

STATE

ENTERED WHEN

WHAT HAPPENS

RED_LINE_STOPPED

A wide red rectangle low in the frame

Full stop for **2 s**, then pull away. Detection is suppressed for **5 s**.

INTERSECTION_MANEUVER

A stop line at a planned intersection

Runs the turn A* planned, then hands control back.

DUCK_STOPPED

A duck confirmed on 3 frames

Stop and watch. Resume if it leaves; commit to a go-around if it settles.

DUCK_OVERTAKE

The duck did not move

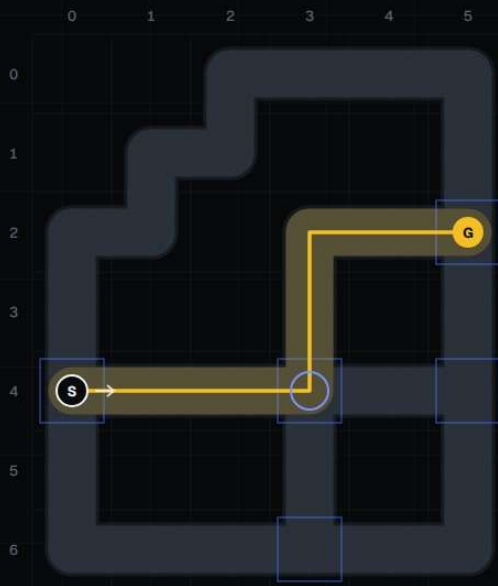
Re-point the PD controller at the yellow line for **3 s** to swerve past.

GOAL_REACHED

The dead-reckoning timer expires

Wheels stopped, run complete.

The track, and the route the planner chose



`tum_map.yaml` describes the track as a grid of tiles, each with a kind and an orientation. A* searches it and returns one route.

Grid	6 × 7 tiles
Start tile, heading	(0, 4) facing E
Goal tile	(5, 2)
Route length	8 tiles
Intersections crossed	1 - at (3, 4)
Manoeuvre required	LEFT turn

Start and goal are 3-way tiles themselves; the count is what the robot drives through.

■ PLANNED ROUTE ■ STOP-LINE INTERSECTION ■ DRIVABLE ROAD

01

Lane following

Vision-based steering that keeps the robot centred, and keeps it there when the lane briefly disappears.



Finding the lane in HSV

Each frame is converted from BGR to **HSV**, which separates hue from brightness and survives the lab's lighting far better than RGB. Three masks per frame: yellow, white, red.

Region of interest

Only the bottom **40%** of the frame: the road just ahead.

Yellow mask

Hue **10–40**, saturation and value ≥ 70 .

White mask

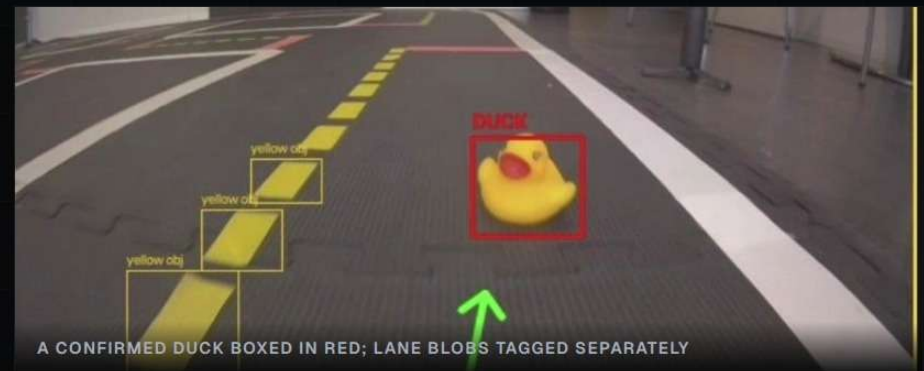
Saturation ≤ 45 , value ≥ 150 : the painted edge, not the floor.

Centroid

The horizontal image moment of each mask. Too little support and it is discarded.

Lane centre

Midpoint of the two, or **25%** of the frame width off a single line.



The lane has a left side and a right side

A camera always sees the same left-to-right layout, so each colour is only searched where it belongs. Search the whole width and the detector can lock onto the **wrong-side** line on a curve.

YELLOW: CENTRE LINE

70%

OF THE FRAME WIDTH, IN FROM
THE LEFT EDGE, YELLOW ONLY

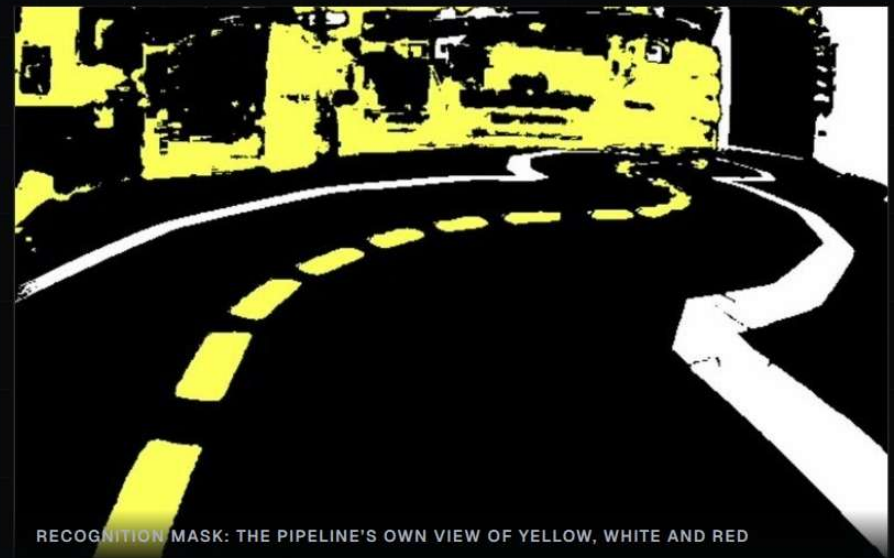
WHITE: OUTER EDGE

30%

OF THE FRAME WIDTH, THEN
RIGHTWARDS, WHITE ONLY

| **Minimum lane width, 20 px.** A white line too close to the yellow one is noise, not a lane edge.

| **Neither line found?** The lane counts as lost for that frame.



RECOGNITION MASK: THE PIPELINE'S OWN VIEW OF YELLOW, WHITE AND RED

A PD controller keeps it centred

$$\text{turn} = - (K_P \cdot e + K_D \cdot (e - e_{\text{prev}}))$$

e = the lane centre's offset from the image centre, normalised to ± 1 . Turn rate is clamped to ± 6.0 rad/s.

Proportional: K_P 4.0

Reacts to how far off-centre the robot is right now.

Speed scaling

Base 0.10, floor 0.05. Speed falls with the **square** of the error.

Derivative: K_D 2.0

Reacts to how fast that error is changing, damping oscillation.

Lost-lane recovery

Keep the last command, decayed **0.85×** per frame, for **15** frames. Then stop.

02

Intersection navigation

Route planning over the track's tile graph, and the manoeuvres that carry the robot through a turn.

From painted tiles to a searchable graph

Every tile kind defines which compass directions its road opens to. A **straight/E** opens East and West; a **4way** opens all four.

```
# two tiles are connected only if
# each one's opening faces the other
if nk in OPENINGS and OPPOSITE[d] in OPENINGS[nk]:
    adj[node].append(((ni, nj), d))
```

That reduces 42 cells to **30 drivable nodes**: the graph A* searches. The 12 grass cells never enter it.

TILE KINDS IN THE MAP

straight · N/S or E/W	2 openings
curve_left, curve_right	2 openings
3way_left	3 openings
4way	4 openings
grass	not drivable

WHY THIS MATTERS

A 3way or 4way tile is the only place a real stop line exists. Marking them lets the planner reason about a route's *cost*, not just its length.

Weighted A*: fewest intersections, not fewest tiles

```
# cost to enter a tile
edge_cost = 1.0
if nb != goal and is_intersection(nb):
    edge_cost += 1000.0
```

Crossing a stop line costs a full stop, a wait and a turn. A **+1000** penalty dwarfs any tile-count difference here, so the planner takes the route with the **fewest intersections** and uses tile count only to break ties.

Admissible heuristic. Manhattan distance never overestimates the remaining cost, which is the condition A* needs.

Heading constraint. The first move must match the robot's actual facing, so a route never opens with a blind U-turn.

ROUTE RETURNED FOR THIS RUN

```
(0,4) → (1,4) → (2,4) → (3,4)
→ (3,3) → (3,2) → (4,2) → (5,2)
```

Tiles traversed	8
Intersections en route	1
Decision at (3,4)	TURN LEFT
Heading in → out	E → N

Walking the route tile by tile, the heading in versus the heading out decides straight, left or right.

Executing the turn

LEFT & STRAIGHT: OPEN LOOP

Short pre-planned (speed, turn-rate, duration) sequences. They can end early once the camera confirms a valid exit lane.

LEFT (0.08, 0.00, 0.8S) → (0.10, 1.20, 1.8S) →
(0.07, 0.00, 1.0S)
STRAIGHT (0.09, 0.00, 3.0S)

RED STOP LINE

Red is searched in the bottom **40%**. A flat rectangle spanning **20%** of the width counts. Stop **2 s**, then a **5 s** cooldown.

RIGHT: CLOSED LOOP

The robot hugs the outer white line at **78%** of the image width, reacting to the camera every frame.

GAIN 2.0 · CLAMP 2.2 · TIMEOUT 6.0S

DEAD RECKONING TO THE GOAL

After the last turn, remaining time is estimated at **4.5 s per tile** and **2.25 s** for the last. Time stopped is credited back.



DUCK3 HELD AT A RED STOP LINE ON THE LAB TRACK

03

Object detection & avoidance

Recognising a duck on the road, and routing around it without leaving the lane-following loop.

A duck is yellow. So is the centre line.

Colour alone cannot separate them, so every yellow contour goes through a **shape classifier** first.

01 Area

Between **2,000** and **160,000** px.

02 Aspect ratio

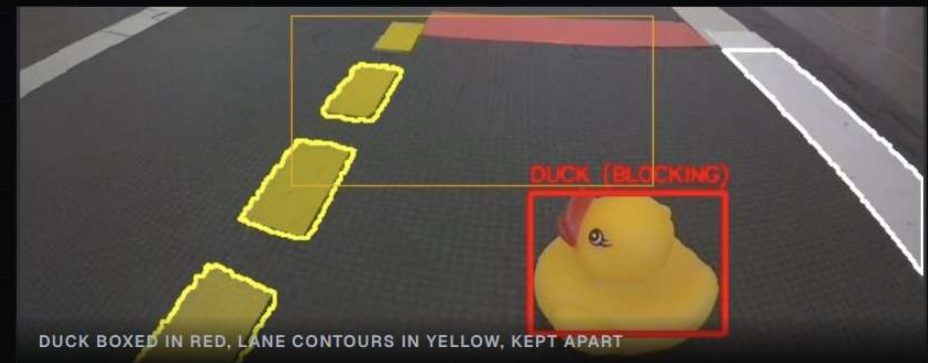
Duck-shaped, loosened above **40,000** px where a close duck is clipped.

03 Shape

Solidity, rectangularity and a polygon approximation all reject the straight edges of a painted line.

04 Then it is erased

Its bounding box is blanked from the yellow channel before lane detection.



Stop, watch, then go around

The window is a fixed three seconds, but the steering inside it is **closed-loop**, reacting to the camera every frame. Unlike a blind timed swerve it cannot overshoot.

01

Confirm

Seen on **3** consecutive frames at $\geq$ **3,500 px**, so a glitch never stops the robot.

02

Stop & watch

Full stop. A shift of **22 px** or **30%** in area resets the settle timer.
Minimum **2 s**, cap **10 s**.

03

Resolve

Four clear frames and the path is free. The robot simply resumes.

04

Go around

Otherwise **3 s** with the PD controller re-pointed at the yellow line, at **0.072** speed and a ± 2.4 clamp.

TRIGGER AREA 3,500 PX

SETTLE 2-10 S

GO-AROUND 3.0 S

RE-TRIGGER COOLDOWN 2.5 S

BETWEEN EPISODES 8 S

After a go-around the 8 s cooldown is cut to 2.5 s, so the duck just passed can't re-trigger it while a new one ahead still can.

04

A negative result

What we tried before the classical pipeline, and why it did not make the cut.

Reinforcement learning never reached the road

183M

SIMULATION STEPS TRAINING A
PPO AGENT, STABLE-BASELINES3

Algorithm	PPO
Framework	stable-baselines3
In simulation	Lane following learned
On duck3	Persistent circular motion

The agent learned to follow lanes inside the simulator. On the real robot it drove in circles: the **sim-to-real gap** in camera imagery, dynamics and lighting that the policy had never seen.

WHY WE REPORT IT

A useful negative result, not a failure to hide. It is what pushed us to a pipeline we can inspect, tune and explain from first principles.

We removed a working U-turn manoeuvre for the same reason: this route never needs one.

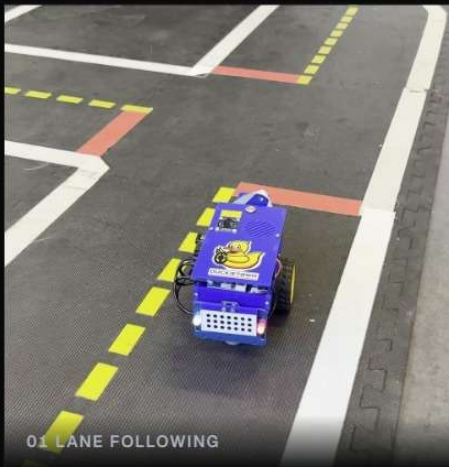
05

Results & engineering

What the system does on the track, how it is operated, and where it falls short.



One uninterrupted run



duck3 drives start to goal repeatedly:
holding the lane, halting at red lines,
taking the planned left turn, and steering
around a duck in its path.

~30

FRAMES PER SECOND
END TO END

6

STATES IN THE
DRIVING LOOP

1

PYTHON PROCESS,
NO ROS NODE DEPLOYED

Operating it, and failing safe

LIVE DASHBOARD

A Flask server on **:5000** streams the annotated feed, the mask, telemetry and the A* route map, from the same process that drives the robot.

WATCHDOG: FAIL SAFE

A background thread stops the wheels if no camera frame arrives for **0.4 s**. Perception hangs, the robot halts.

ROUTE SETUP

The operator picks start tile, goal tile and heading on a rendered map. duck3 stays stopped until a mode is chosen.

MANUAL OVERRIDE

Keyboard control is always available, and **keyboard_drive.py** drives the wheels directly for calibration.



OPERATOR DASHBOARD: LIVE FEED, RECOGNITION MASK, TELEMETRY, ROUTE MAP

What it does not do yet

01

Lighting sensitivity

Fixed HSV thresholds, tuned to the TUM lab. Different lighting needs re-tuning by hand.

02

Hand-authored map

The tile map is written by hand for this track. Nothing surveys or calibrates it, so a new track means a new file.

03

Duck shape assumption

Tuned to the specific yellow duck used in testing. Other sizes or colours may be missed.

04

No localisation

Position comes from counting stop lines, then timing. A missed detection desynchronises the plan.

WHERE WE WOULD GO NEXT

| Auto-tune the thresholds

Calibrate HSV against the current scene at start-up.

| Generalise the detector

Extend the classifier past one colour, one size, one kind of obstacle.

| Close the loop on position

Confirm which tile the robot is actually on.

IN CLOSING

The pipeline is classical on purpose

HSV thresholding, a PD controller and a graph search, not a learned policy. That trade is what got duck3 across town.

| Inspectable

Every decision traces back to a threshold, a gain or a graph edge.

| Tunable

Wrong behaviour meant changing a number, not retraining.

| Repeatable

Same route, same turn, same duck response, run after run.

Thank you

Yahia Taha · Ahmad Kaihan Saba · Đorđe Vidaković

TECHNICAL UNIVERSITY OF MUNICH · CAMPUS HEILBRONN

Hardware and software platform developed by the Duckietown Project (duckietown.org)